

# MAPS, HASHMAPS & FUNCTIONAL COMBINATORS IN SCALA



by  
**Dr M Rajasekhara Babu**

School of  
Computer Science and Engineering  
<https://www.rajababu.org>



# VIT<sup>®</sup>

## Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Vellore-632014, Tamil Nadu, India





# Learning Objectives

01

## **Understand Maps & HashMaps**

Store and retrieve key-value pairs in Scala.

02

## **Apply map() Combinator**

Transform collections using functional programming.

03

## **Handle Collisions**

Understand HashMap collisions and resolution strategies.

04

## **Merge HashMaps**

Combine two HashMaps with conflict resolution logic.

05

## **Real-World Applications**

Implement student grade management using HashMaps.



# What is a Map?

- A Map stores information as **key-value pairs** — like a dictionary where every key maps to exactly one value.

**Student Name → Grade**

**Employee ID → Salary**

**Product ID → Price**

**Email → Password**



# Creating & Accessing a Map

## Creating a Map

```
val grades = Map(  
    "Alice"    -> 85,  
    "Bob"      -> 90,  
    "Charlie"  -> 88 )
```

## Accessing Values

```
println(grades("Alice"))  
// Output: 85
```

## Checking Key Existence

```
println(grades.contains("Bob"))  
// Output: true
```

## Why Use Maps?

- Suppose we have 100 students.
- Without a Map, we'd need 100 separate variables.
- With a Map, retrieval is instant using a key.

**// Without Map**

**val grade1 = 85**

**val grade2 = 90**

**// ...100 variables**

**// With Map**

**grades("Alice")**

**grades("Bob")**

Maps provide fast retrieval using keys  
— no loops needed.



# Mutable HashMap

Unlike immutable Maps, a HashMap allows adding, updating, and removing entries at runtime.

## Import & Create

```
import scala.collection.mutable.HashMap  
val grades = HashMap[String, Int]()
```

## Adding Entries

```
grades += ("Alice" -> 85)  
grades += ("Bob" -> 90)  
grades += ("Charlie" -> 88)  
// Output:  
// HashMap(Alice -> 85,  
//          Bob -> 90,  
//          Charlie -> 88)
```

## Map vs HashMap

### Map — Immutable

```
val m = Map("A" ->  
1)
```

Cannot be changed after creation.

### HashMap — Mutable

```
val hm = HashMap("A"  
-> 1)
```

Entries can be added, updated, or removed.





# Functional Combinator: map()

map() applies a function to *every element* in a collection and returns a new collection with the results.

## Basic Example

```
val grades = List(80, 85, 90)
val updated = grades.map(x => x + 5)
println(updated)
// Output: List(85, 90, 95)
```

## Real Use — Bonus Marks

```
val bonusGrades = grades.map(_ + 10)
// Output: List(90, 95, 100)
```

## How It Works

### 1 Input Collection

Start with a List of values.

### 2 Apply Function

The function runs on each element.

### 3 New Collection

A transformed List is returned.



# Duplicate Keys in HashMap

What happens when the **same key is inserted twice**? The old value is silently replaced by the new one.

## Example

```
val grades = HashMap("Alice" -> 85)
grades += ("Alice" -> 95)
println(grades)
// Output: HashMap(Alice -> 95)
```

## Observation

| Old Value  | New Value  |
|------------|------------|
| Alice → 85 | Alice → 95 |

- The HashMap keeps only the **most recently inserted value** for any given key.
- This is key behavior to remember when updating records.

A **collision** occurs when multiple keys hash to the same internal storage bucket.

## Example

Alice --> Bucket 4

David --> Bucket 4

// Both keys attempt to  
// occupy the same bucket!

- Internally, a hash function converts each key to a bucket index.
- Different keys can produce the same index — this is a collision.

## Collision Handling Techniques

### Chaining <sup>1</sup>

Store multiple values in a List at the same bucket. Easiest for beginners.

### Open Addressing <sup>2</sup>

Store the data in the next free bucket location.

### Double Hashing <sup>3</sup>

Apply a second hash function to find an alternate bucket.



# Merging Two HashMaps

When merging, a **conflict** arises if the same key exists in both maps. We resolve it by keeping the higher grade using `Math.max()`.

## Map 1

```
Alice    -> 85
Bob       -> 90
Charlie  -> 88
```

## Map 2

```
Bob       -> 92
David     -> 80
```

**Conflict: Bob exists in both!**

Which value do we keep — 90 or 92?

```
Math.max(90, 92)
// Result: 92
```

## Merged Result

```
Alice -> 85
Bob   -> 92
```



# Complete Program: mergeMaps()

```
import scala.collection.mutable.HashMap

object HashMapDemo {
  def mergeMaps(
    map1: HashMap[String, Int],
    map2: HashMap[String, Int]
  ): HashMap[String, Int] = {
    val merged = HashMap[String, Int]()
    for ((key, value) <- map1)
      merged(key) = value
    for ((key, value) <- map2) {
      if (merged.contains(key))
        merged(key) =
          Math.max(merged(key), value)
      else
        merged(key) = value
    }
    merged
  }
}
```

The function iterates over both maps. For each key in Map 2, it checks if the key already exists in the merged result — if so, it keeps the higher value using `Math.max()`.

Chaining stores multiple values for the same key in a `List[Int]`, preserving all entries rather than overwriting.

## addWithChaining() Method

```
def addWithChaining(  
  map: HashMap[String, List[Int]],  
  key: String,  
  value: Int  
): Unit = {  
  if (map.contains(key))  
    map(key) = map(key) :+ value  
  else  
    map(key) = List(value)  
}
```

## Usage

```
addWithChaining(collisionMap, "Alice", 85)  
addWithChaining(collisionMap, "Alice", 95)  
addWithChaining(collisionMap, "Bob", 90)
```

```
// Output:  
// HashMap(  
//   Alice -> List(85, 95),  
//   Bob   -> List(90)  
// )
```

Alice's two grades are both preserved in a List — no data is lost.

## Grades1 (after duplicate key)

```
HashMap(  
  Alice    -> 95,  
  Bob      -> 90,  
  Charlie  -> 88  
)
```

## Merged Grades

```
HashMap(  
  Alice    -> 95,  
  Bob      -> 92,  
  Charlie  -> 88,  
  David    -> 80  
)
```

## Collision Map (Chaining)

```
HashMap(  
  Alice -> List(85, 95),  
  Bob   -> List(90)  
)
```

📱 Notice that Alice appears in both the merged map (with her highest grade 95) and the collision map (with all her grades preserved as a List).

1

## Step 1 — Duplicate Key

Alice → 85 then Alice → 95.  
Final: Alice → 95 (overwritten).

2

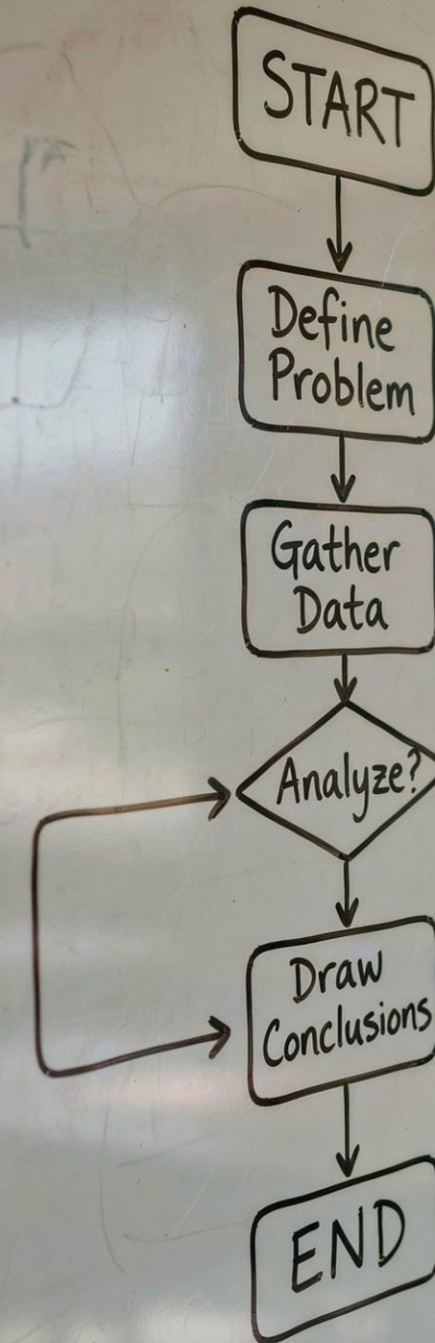
## Step 2 — Merge Conflict

Bob → 90 vs Bob → 92.  
Math.max(90, 92) → Bob → 92.

3

## Step 3 — Chaining

Alice → 85 then Alice → 95 stored as  
Alice → List(85, 95).



1

## Activity 1 – Create & Print

Create a HashMap:

```
"Java" -> 4,  
"Scala" -> 5,  
"Python" -> 5
```

Print all keys and values.

2

## Activity 2 – Merge Maps

Merge Map1 (A→70, B→80) and Map2 (B→90, C→75). Select the higher value for conflicts.

3

## Activity 3 – Chaining

Implement chaining for Course → Marks. Store multiple marks for the same course using a List.

# Practice Program Problem Statement

Handling collisions and resolving conflicts in HashMap's: Create a HashMap with keys representing student names and values representing their grades, then insert multiple entries including a duplicate key with a different grade. Write a function to merge two HashMap's, resolving conflicts by taking the higher grade. Finally, write a function to handle collisions by chaining, using lists to store multiple values for a single key.



## Maps & HashMaps

Key-value pair storage. Maps are immutable; HashMaps are mutable and support runtime updates.



## map() Combinator

Applies a function to every element in a collection, returning a new transformed collection.



## Collisions

Occur when two keys hash to the same bucket. Resolved via chaining, open addressing, or double hashing.



## Conflict Resolution

When merging HashMaps, use `Math.max()` to keep the higher value for duplicate keys.

## ✓ **Maps & HashMaps**

Key-value pair storage in Scala.

## ✓ **Mutable Operations**

Add, update, and remove entries in a HashMap.

## ✓ **map() Combinator**

Functional transformation of collections.

## ✓ **Duplicate Keys**

Latest value overwrites the previous one.

## ✓ **Collision Handling**

Chaining with Lists preserves all values.

## ✓ **Map Merging**

Conflict resolution using `Math.max()`.



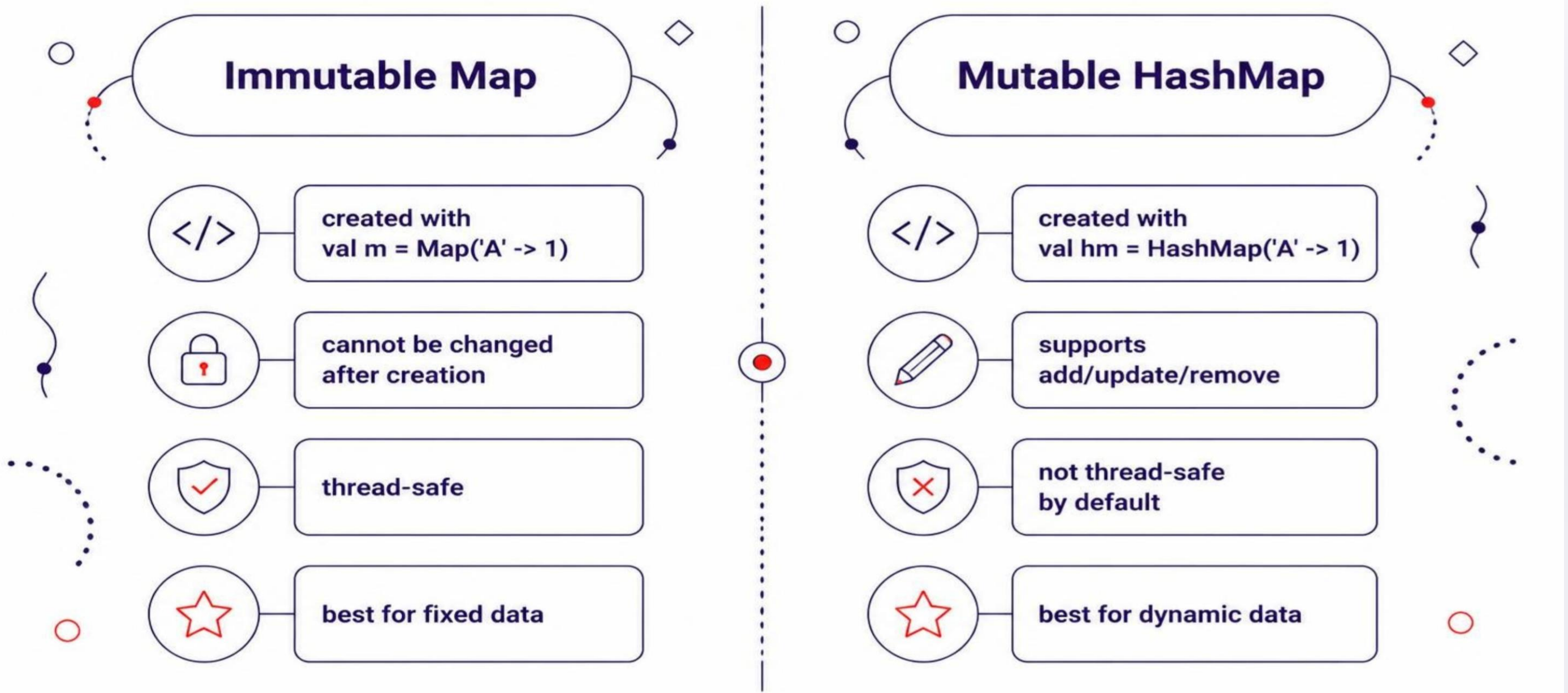
# Assignment

- 1 Employee Salaries**  
Create a HashMap storing employee names and salaries.
- 2 Merge Salary Maps**  
Merge two salary HashMaps and keep the higher salary for conflicts.
- 3 Collision Handling**  
Implement a collision-handling HashMap using `List[Double]`.
- 4 Apply map()**  
Use `map()` to add 5 marks to every grade in a list.
- 5 Grade Management**  
Write a full program to manage student grades using HashMap and conflict resolution.



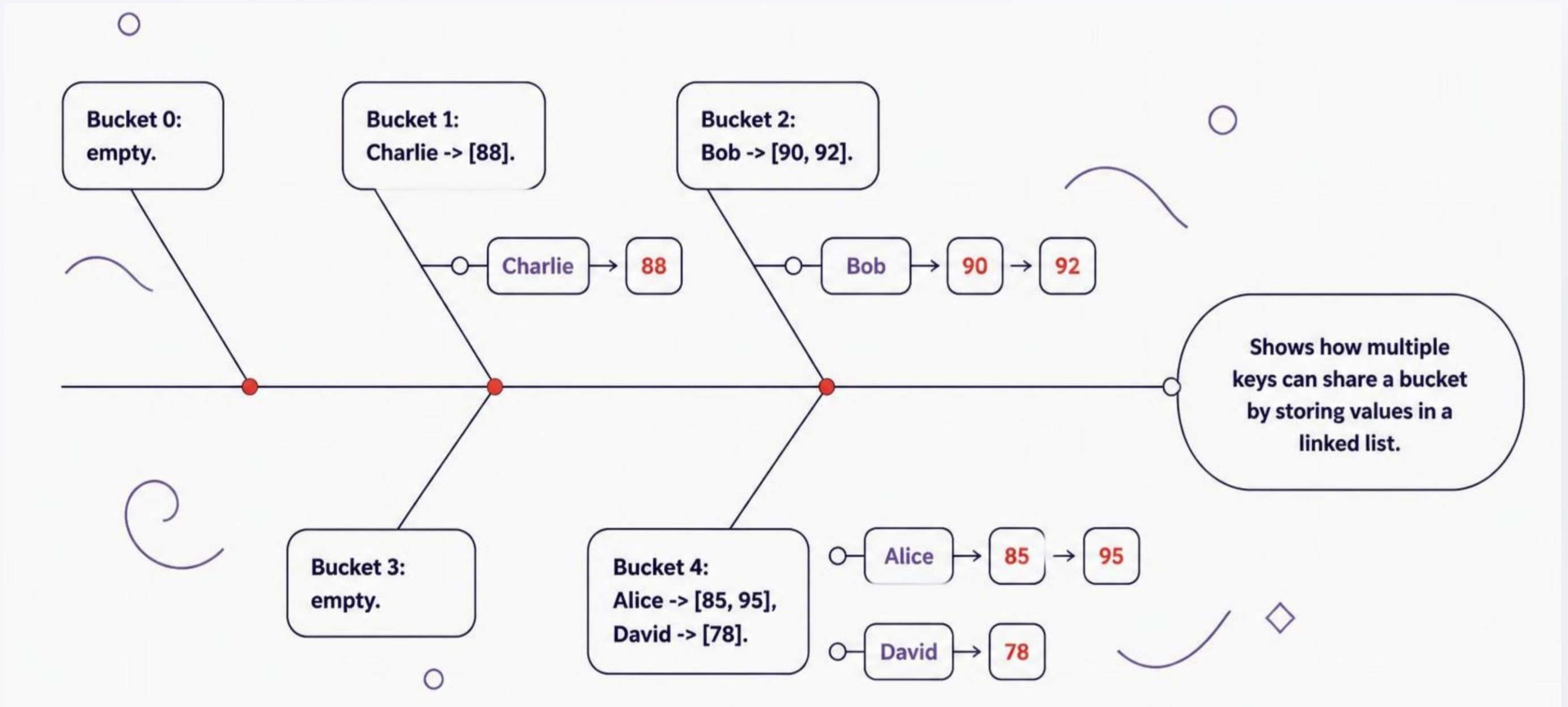
# Immutable Map vs Mutable HashMap

Choose Map when data is fixed;  
choose HashMap when you need to modify entries at runtime.



# Collision Resolution: Chaining Deep Dive

Chaining appends new values to an existing List at the same bucket, ensuring no data is lost during a collision.



# Keep Coding in Scala!

You now have the tools to build real-world data management systems using Maps, HashMaps, functional combinators, and collision-safe storage strategies.



**Maps**



**HashMaps**



**map()**



**Chaining**



**Merging**